

These two functions are the crux of the optimization analysis: `Stratify()` and `summary.strata()`. `Stratify()` takes two data frames. The first, 'x', is the raw data frame where the count/sample (e.g., number of individuals in a sample) information is located. This data frame needs to include, for each sample, the strata name and the count ('species' in the function arguments). The original BioSurvey2 package assumed that the data would be coming from a fisheries tow and would have counts of multiple species for each sample, but is only designed to work with one species at a time. So, the species would have a column header (e.g., AtlanticCod), and the rows for that column would contain the number of individuals of that species were in that sample. The second data frame, 'strata.group', includes the strata name and the theoretical maximum number of samples (NH) that could have been taken from that strata (e.g., for the oyster survey, each sample was a 1 m² grab, and so the maximum number of samples would have been the area of the reef in m²). Obviously, the strata names need to be identical between the two data frames. The function also allows you to specify a subset of your data from 'x' for processing. I have [commented](#) the functions as best I can based on the [manual](#) that Stephen Smith published for the SPLUS program that preceded the R package.

Some useful definitions from that manual:

n_h = the number of hauls or sets sampled in stratum h ($h = 1, \dots, L$),

n = $\sum_{h=1}^L n_h$, the total number of sets sampled.

N_h = the total number of sets sampled,

N = $\sum_{h=1}^L N_h$, the total number of possible sets in the survey area.

f_h = $\frac{n_h}{N_h}$, the sampling fraction in stratum h .

W_h = $\frac{N_h}{N}$, the proportion of the area in stratum h .

y_{hi} = the number of fish caught in set i and stratum h ,

$\bar{y}_h$ = $\sum_{i=1}^{n_h} \frac{y_{hi}}{n_h}$, the estimated mean abundance in stratum h ,

s_h^2 = $\sum_{h=1}^L \frac{y_{hi} - \bar{y}_h}{n_h - 1}$, the estimated variance in stratum h .

$\bar{y}_{st}$ = $\sum_{h=1}^L W_h \bar{y}_h$, the stratified mean abundance.

```
> Stratify
function (x, strata.group, strata.name, species, Subset)
{
  m <- match.call()
  if (!missing(Subset)) {
    x <- x[Subset, ]
  }
  # If 'Subset' is missing from the input, proceed with the full dataset, else subset the data

  obj.Strata <- x[[strata.name]]
  # Create a vector of the strata names in 'x'

  s.group <- is.element(strata.group$Strata, unique(obj.Strata))
  # Selects the strata in 'x' for which there is strata information in the 'strata.group' data frame

  s.group.Strata <- strata.group$Strata[s.group]
  # Subset the 'strata.group' strata names to only include records for which there is count
  # information
}
```

```

s.group.NH <- strata.group$NH[s.group]
# Create a vector of the NH data

s.obj <- is.element(obj.Strata, s.group.Strata)
# Create a vector of the strata names from obj.Strata that are in s.group.Strata

x <- x[s.obj, ]
# Subset 'x' to get only the rows for which all required data exists

species <- x[[species]]
# Create a vector of the counts

x <- x[s.obj, ]
# Redundant

obj.Strata <- obj.Strata[s.obj]
# Subset 'obj.strata' to get only the rows for which all required data exists

yhi <- split(species, obj.Strata)
# splits the 'x' dataset into a list of counts for each strata

nh <- as.vector(sapply(yhi, length))
# Creates a vector of the number of samples within each strata

yhi <- yhi[nh != 0]
# Removes all records with no counts

nh <- nh[nh != 0]
# Removes all records with no counts

res <- list(yhi = yhi, Strata = s.group.Strata, Nh = s.group.NH,
           Wh = s.group.NH/sum(s.group.NH), nh = nh, call = m)
# Create a list of the relevant information: yhi (count data in each strata), strata names,
# maximum number of samples (Nh) within each strata, the proportion of the total possible samples
# across all strata within in each strata (Wh), and the actual number of samples in each
# strata (nh)

class(res) <- "strata"
# set the class of the list to type "strata"

return(res)
# return the list
}

```

At this point, we have what amounts to a joined table of all of the necessary information in list format. This list will be passed to `summary.strata()` to analyze the things that we're interested in (efficiency and optimization).

```

> summary.strata
function (object, alpha.t = 0.05, effic = FALSE, nopt = FALSE,
  ...)
{
  if (!inherits(object, "strata"))
    stop("Not a legitimate strata object")
  # Stops the analysis if the object passed to "summary.strata()" isn't of class "strata"

  yh <- as.vector(sapply(object$yhi, mean))
  # creates a vector of the mean count within each strata

  yst <- sum(object$Wh * yh, na.rm = TRUE)
  # calculate the stratified mean abundance

  sh <- as.vector(sapply(object$yhi, var))
  # calculate the variance within each strata

  se.yst <- sqrt(sum((((object$Nh * (object$Nh - object$nh))/sum(object$Nh)^2) *
    sh)/object$nh, na.rm = TRUE))
  # calculate the stratified standard error

  ah <- (object$Nh * (object$Nh - object$nh))/object$nh
  # Honestly not certain here. It's used for calculating degrees of freedom below

```

```

df.yst <- (sum(ah * sh, na.rm = TRUE)^2)/(sum(((ah * sh)^2)/(object$nh -
1), na.rm = TRUE))
# calculate the degrees of freedom

ci.yst <- yst + (c(qt(alpha.t/2, df.yst), -qt(alpha.t/2,
df.yst)) * se.yst)
# calculate the CI around the mean

res <- list(yst = yst, se.yst = se.yst, Yst = yst * sum(object$Nh),
df.yst = df.yst, alpha = alpha.t, ci.yst = ci.yst)
# put everything into a list

if (effic == TRUE) {
# If you want to calculate the efficiency, do the following

N <- sum(object$Nh)
# Total possible samples

n <- sum(object$nh)
# Total actual samples

vran <- ((N - n)/(n * N)) * sum(object$Wh * sh, na.rm = TRUE)
# calculate the variance of a randomly distributed survey design

effic.str <- ((N - n)/(n * (N - 1))) * (sum(object$Wh *
(yh - yst)^2) - sum((object$Wh * (1 - object$Wh) *
sh)/object$nh, na.rm = TRUE))
# calculate the efficiency of the current strata (improvement in variance)

vran <- vran + effic.str
# calculate the amount of unexplained variance

effic.alloc <- sum(((1/n) - (object$Wh/object$nh)) *
object$Wh * sh, na.rm = TRUE)
# calculates the allocation efficiency

effic.str <- (100 * effic.str)/vran
# Percent efficiency of the stratified approach relative to the random variance

effic.alloc <- (100 * effic.alloc)/vran
# Percent efficiency of the effort allocation relative to the random approach

min.var <- ((1/n) * sum(object$Wh * sqrt(sh), na.rm = TRUE)^2) -
sum(object$Wh * sh, na.rm = TRUE)/N
# The theoretical minimum variance possible within the data

res <- c(res, list(effic.alloc = effic.alloc, effic.str = effic.str,
var.ran = vran, max.eff = (100 * (vran - min.var))/vran))
# Put all of the data into a list

}
if (nopt) {
# methods if 'nopt' = TRUE

n.opt.out <- as.data.frame(matrix(NA, length(yh) + 1,
6))
# create an empty data frame

names(n.opt.out) <- c("Strata", "Observed", "Optimal",
"Perc.Increase.Var.opt", "Compromise", "Perc.Increase.Var.comp")
# set the names

n.opt.out$Strata <- c(object$Strata, "Total")
# add strata names plus a line for the Total

n.opt.out$Observed <- c(object$nh, n.tot <- sum(object$nh))
# number of samples per strata plus the total

if (all(object$nh != 1)) {
# if there was more than 1 sample...

n.opt <- (((sqrt(sapply(object$yhi, var)) * object$Wh)/sum(sqrt(sapply(object$yhi,

```

```

    var)) * object$Wh, na.rm = TRUE)) * sum(object$nh))
# The optimal number of samples in a strata is a positive function of both the variance
# of the counts within that strata and the proportional size of the strata relative to the
# rest of the study area.

n.opt <- round(n.opt)
# round the samples to integers

if (n.tot > sum(n.opt)) {
  n.opt[n.opt == max(n.opt)] <- n.opt[n.opt ==
    max(n.opt)] + (n.tot - sum(n.opt))
}
# if there are more samples in the original than the optimal, add the difference to the
# strata with the most recommended samples

if (n.tot < sum(n.opt)) {
  n.opt[n.opt == max(n.opt)] <- n.opt[n.opt ==
    max(n.opt)] - (n.tot - sum(n.opt))
}
# if there are more samples in the optimal than the original, subtract the difference from
# the strata with the most recommended samples

n.opt.out$Optimal <- c(n.opt, sum(n.opt))
# set the final number of optimal samples in the data frame

perc.increase <- (100 * (((object$nh - n.opt)^2)/object$nh))/sum(object$nh)
# calculate the percent increase in strata variance with the new recommended effort

n.opt.out$Perc.Increase.Var.opt <- c(perc.increase,
  sum(perc.increase))
# set the final variance increases

n.comp.ind <- (n.opt <= 1)
# find which strata have <= 1 recommended sample

n.comp.opt <- sum(n.comp.ind)
# how many strata have <= 1 recommended samples

n.opt2 <- n.opt
# create a new optimal vector

n.opt2[n.comp.ind] <- 2
# set all of the strata with <= 1 recommended samples to 2 samples

n.opt2[n.opt2 == max(n.opt2)] <- n.opt2[n.opt2 ==
  max(n.opt2)] - (sum(n.opt2) - sum(n.opt))
# subtract the added compromise samples from the maximum strata

n.opt.out$Compromise <- c(n.opt2, sum(n.opt2))
# set the optimal samples with compromise in the data frame

Perc.Increase.Var.comp <- (100 * (((n.opt2 - n.opt)^2)/n.opt2))/sum(object$nh)
# recalculate the percent change in strata variance for the compromised option

n.opt.out$Perc.Increase.Var.comp <- c(Perc.Increase.Var.comp,
  sum(Perc.Increase.Var.comp))
# set the new compromised variance in the data frame
}
res <- c(res, list(n.opt = n.opt.out))
# make a list
}

options(digits = max(options())$digits - 5, 5))
# output precision to 5 digits

c(res, list(descrip = "Stratified Analysis"))
# return the lists
}

```

Note that the optimal number of samples in a small but highly variable (aggregated) area may be similar to the optimal number of samples in a large but more homogeneous area. Large and variable areas are going to demand more samples than small homogeneous areas.